

Rajnish_Maurya DA6401 - Assignment 1

Write your own backpropagation code and keep track of your experiments using wandb.ai.

Deadline: 10th March 2025, Hard deadline. No extensions will be provided.

[Rajnish Maurya da24m015](#)

Created on March 9 | Last edited on March 17

▾ Instructions

- The goal of this assignment is twofold: (i) implement and use gradient descent (and its variants) with backpropagation for a classification task (ii) get familiar with Wandb which is a cool tool for running and keeping track of a large number of experiments
- This is a **individual assignment** and no groups are allowed.
- Collaborations and discussions with other students is strictly prohibited.
- You must use Python (NumPy and Pandas) for your implementation.
- You cannot use the following packages from Keras, PyTorch, Tensorflow: optimizers, layers
- If you are using any packages from Keras, PyTorch, Tensorflow then post on Moodle first to check with the instructor.

- You have to generate the report in the same format as shown below using wandb.ai. You can start by cloning this report using the clone option above. Most of the plots that we have asked for below can be (automatically) generated using the APIs provided by `wandb.ai`. You will upload a link to this report on Gradescope.
- You also need to provide a link to your GitHub code as shown below. Follow good software engineering practices and set up a GitHub repo for the project on Day 1. Please do not write all code on your local machine and push everything to GitHub on the last day. The commits in GitHub should reflect how the code has evolved during the course of the assignment.
- You have to check Moodle regularly for updates regarding the assignment.

▼ Problem Statement

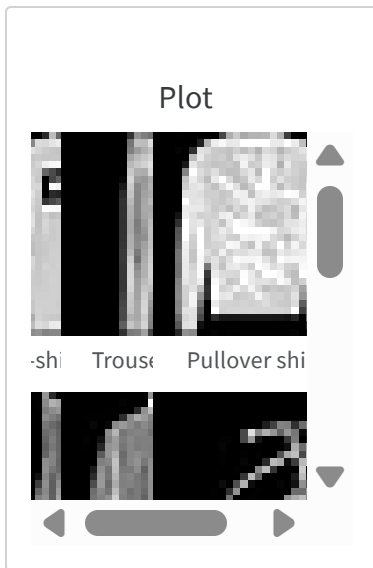
In this assignment you need to implement a feedforward neural network and write the backpropagation code for training the network. We strongly recommend using numpy for all matrix/vector operations. You are not allowed to use any automatic differentiation packages. This network will be trained and tested using the Fashion-MNIST dataset. Specifically, given an input image ($28 \times 28 = 784$ pixels) from the Fashion-MNIST dataset, the network will be trained to classify the image into 1 of 10 classes.

Your code will have to follow the format specified in the `Code Specifications` section.

▼ Question 1 (2 Marks)

Download the fashion-MNIST dataset and plot 1 sample image for each class as shown in the grid below. Use `from`

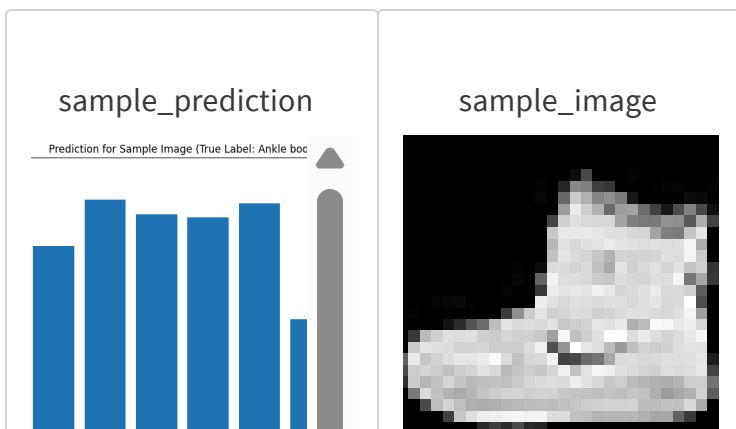
```
keras.datasets import fashion_mnist
```

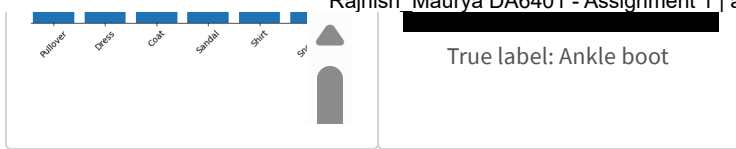
 for getting the fashion
mnist dataset.

▼ Question 2 (10 Marks)

Implement a feedforward neural network which takes images from the fashion-mnist data as input and outputs a probability distribution over the 10 classes.

Your code should be flexible such that it is easy to change the number of hidden layers and the number of neurons in each hidden layer.





▼ Question 3 (24 Marks)

Implement the backpropagation algorithm with support for the following optimisation functions

- sgd
- momentum based gradient descent
- nesterov accelerated gradient descent
- rmsprop
- adam
- nadam

(12 marks for the backpropagation framework and 2 marks for each of the optimisation algorithms above)

We will check the code for implementation and ease of use (e.g., how easy it is to add a new optimisation algorithm such as Eve). Note that the code should be flexible enough to work with different batch sizes.

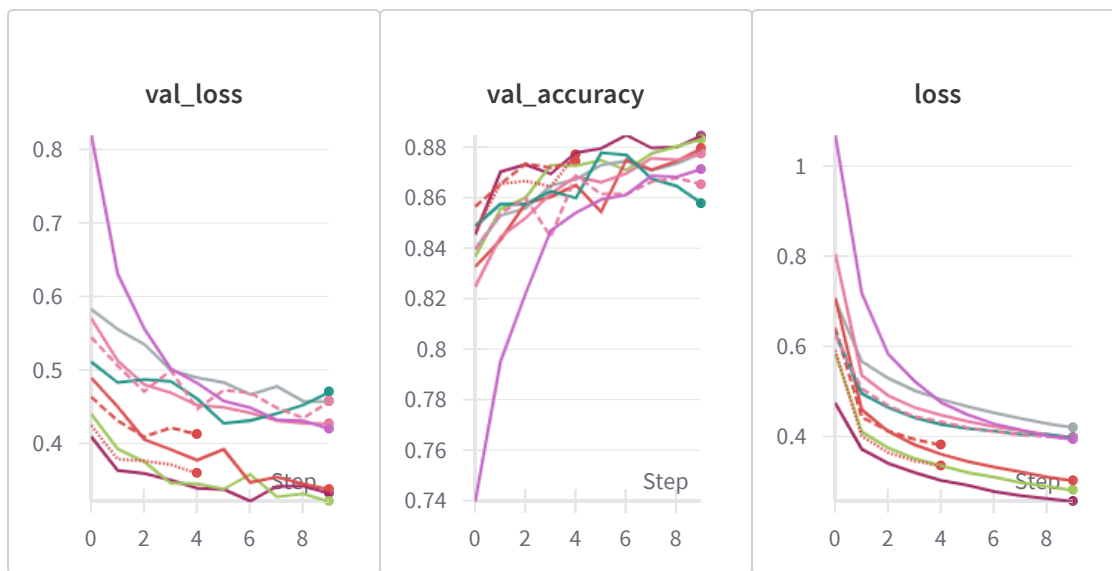
▼ Question 4 (10 Marks)

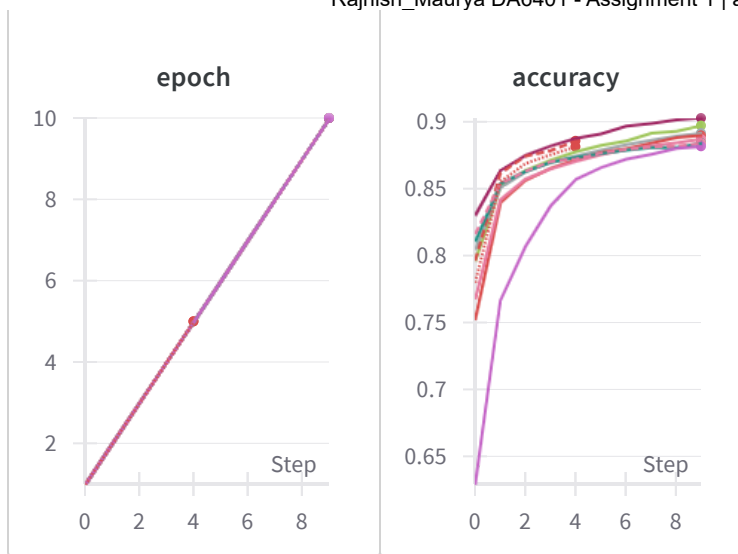
Use the sweep functionality provided by wandb to find the best values for the hyperparameters listed below. Use the standard train/test split of fashion_mnist (use `(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()`). Keep 10% of the training data aside as validation data for this hyperparameter search. Here are some suggestions for different values to try for hyperparameters. As you can quickly see that this leads to an exponential number of combinations. You will

have to think about strategies to do this hyperparameter search efficiently. Check out the options provided by wandb.sweep and write down what strategy you chose and why.

- number of epochs: 5, 10
- number of hidden layers: 3, 4, 5
- size of every hidden layer: 32, 64, 128
- weight decay (L2 regularisation): 0, 0.0005, 0.5
- learning rate: 1e-3, 1 e-4
- optimizer: sgd, momentum, nesterov, rmsprop, adam, nadam
- batch size: 16, 32, 64
- weight initialisation: random, Xavier
- activation functions: sigmoid, tanh, ReLU

wandb will automatically generate the following plots. Paste these plots below using the "Add Panel to Report" feature. Make sure you use meaningful names for each sweep (e.g. hl_3_bs_16_ac_tanh to indicate that there were 3 hidden layers, batch size was 16 and activation function was ReLU) instead of using the default names (whole-sweep, kind-sweep) given by wandb.

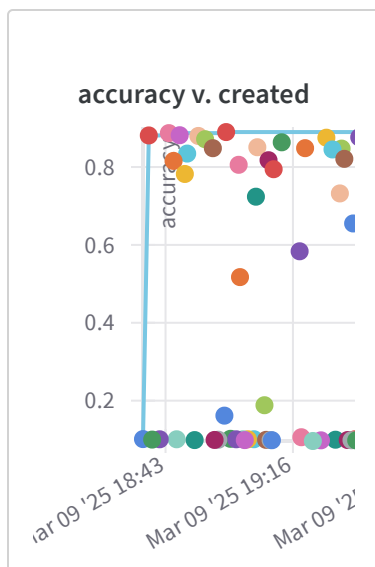




▼ Question 5 (5 marks)

We would like to see the best accuracy on the validation set across all the models that you train.

wandb automatically generates this plot which summarises the test accuracy of all the models that you tested. Please paste this plot below using the "Add Panel to Report" feature



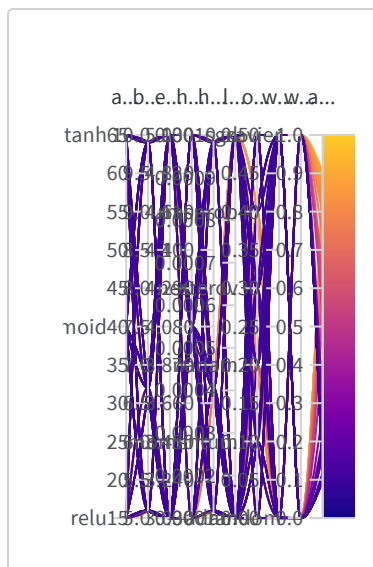
Question 6 (20 Marks)

Based on the different experiments that you have run we want you to make some inferences about which configurations worked and which did not.

Here again, wandb automatically generates a "Parallel co-ordinates plot" and a "correlation summary" as shown below. Learn about a "Parallel co-ordinates plot" and how to read it.

By looking at the plots that you get, write down some interesting observations (simple bullet points but should be insightful). You can also refer to the plot in Question 5 while writing these insights. For example, in the above sample plot there are many configurations which give less than 65% accuracy. I would like to zoom into those and see what is happening.

I would also like to see a recommendation for what configuration to use to get close to 95% accuracy.



Parameter importance with respect to

||| accuracy

Search

◀ ▶

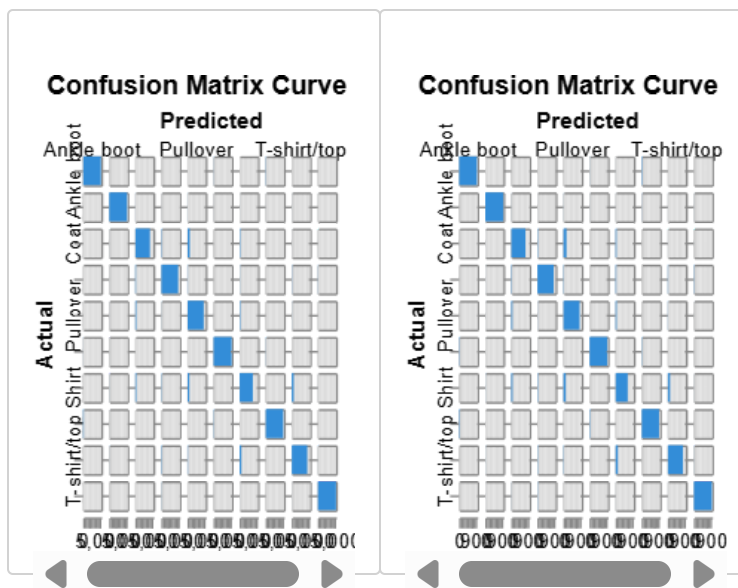
Config para... Importa

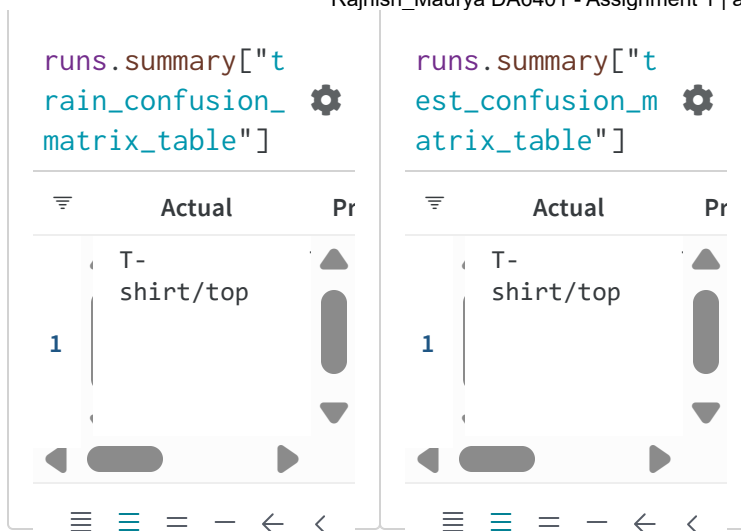
weigl...leacy

◀ ▶

Question 7 (10 Marks)

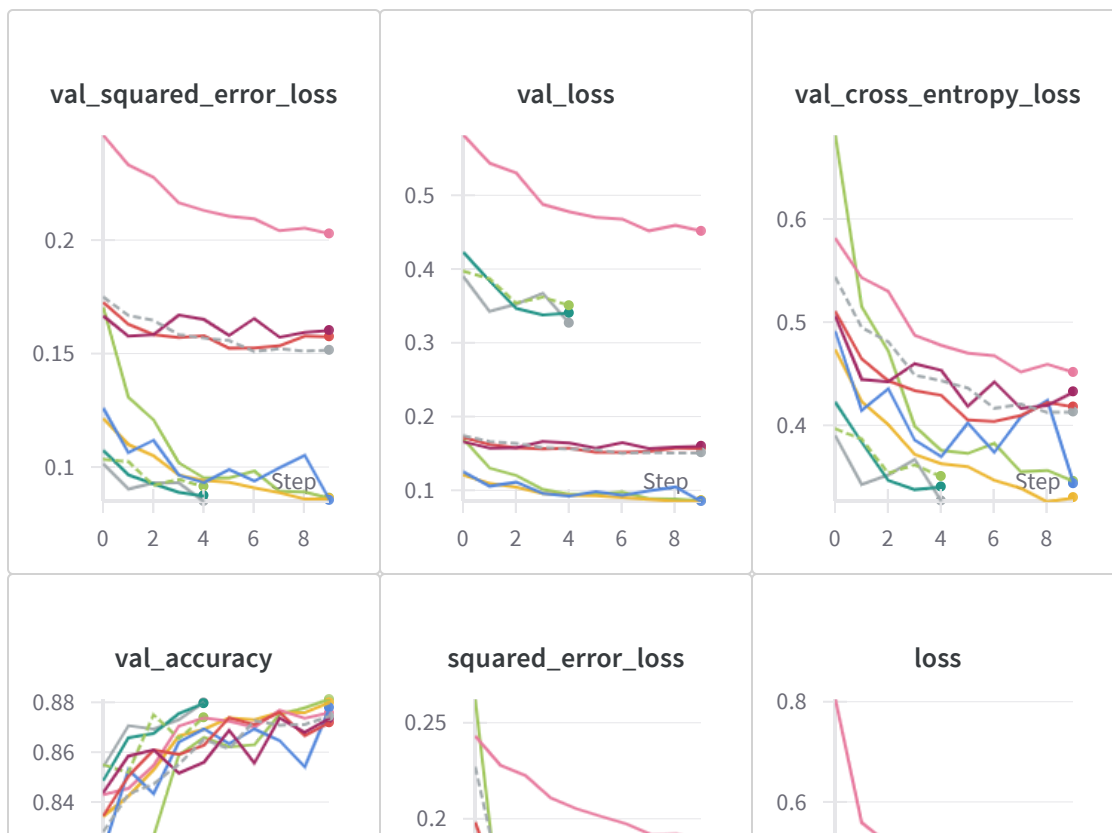
For the best model identified above, report the accuracy on the test set of fashion_mnist and plot the confusion matrix as shown below. More marks for creativity (less marks for producing the plot shown below as it is)

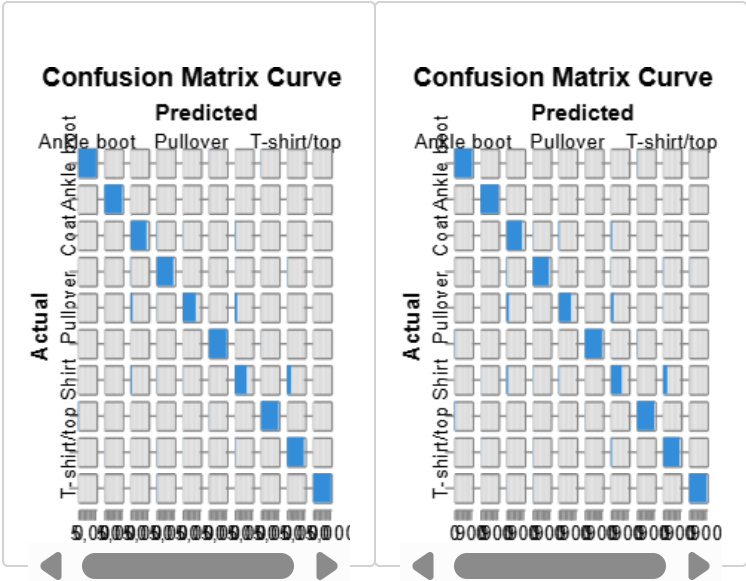
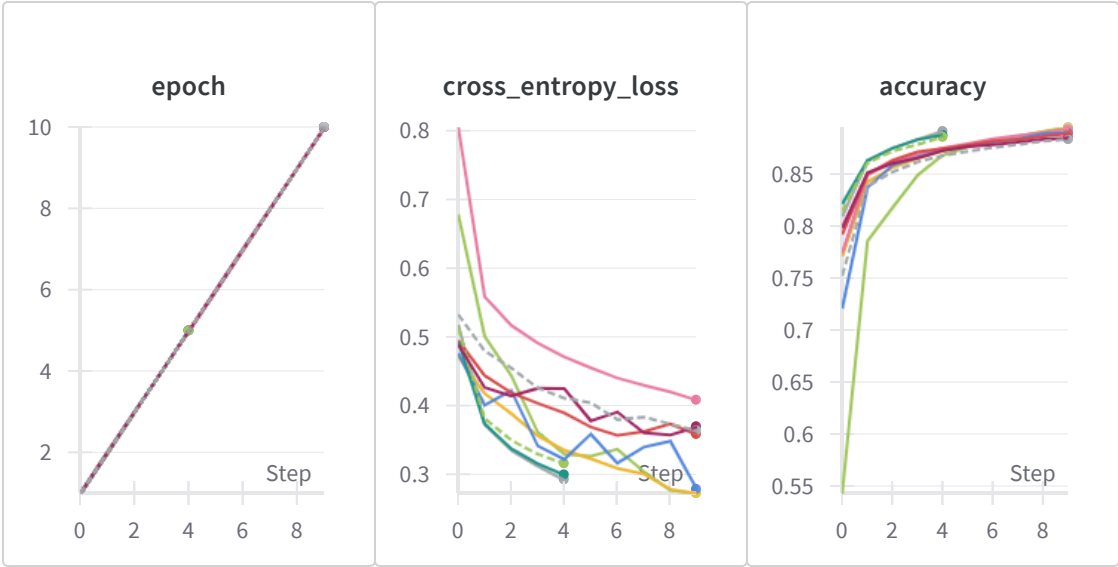
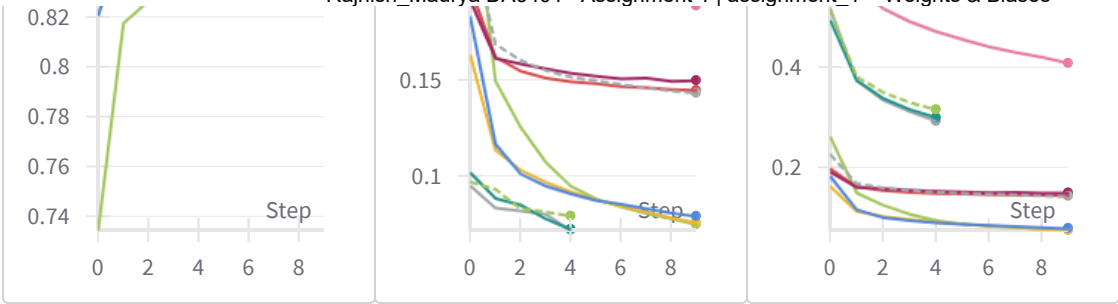


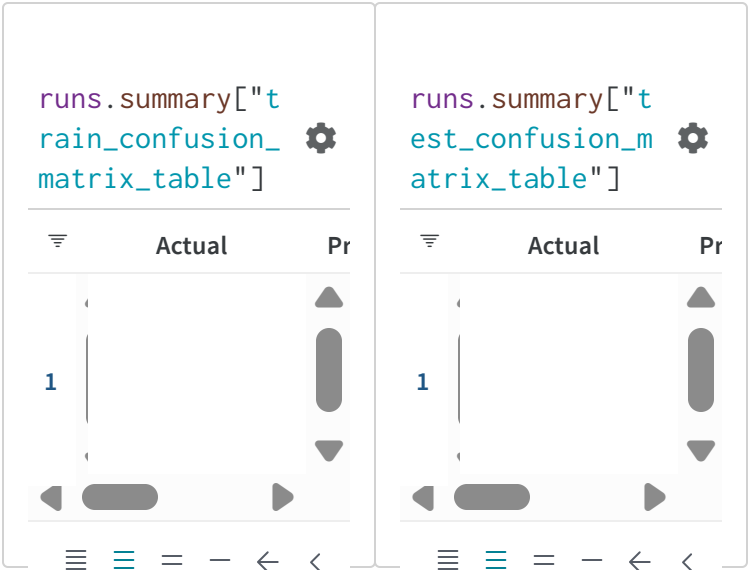


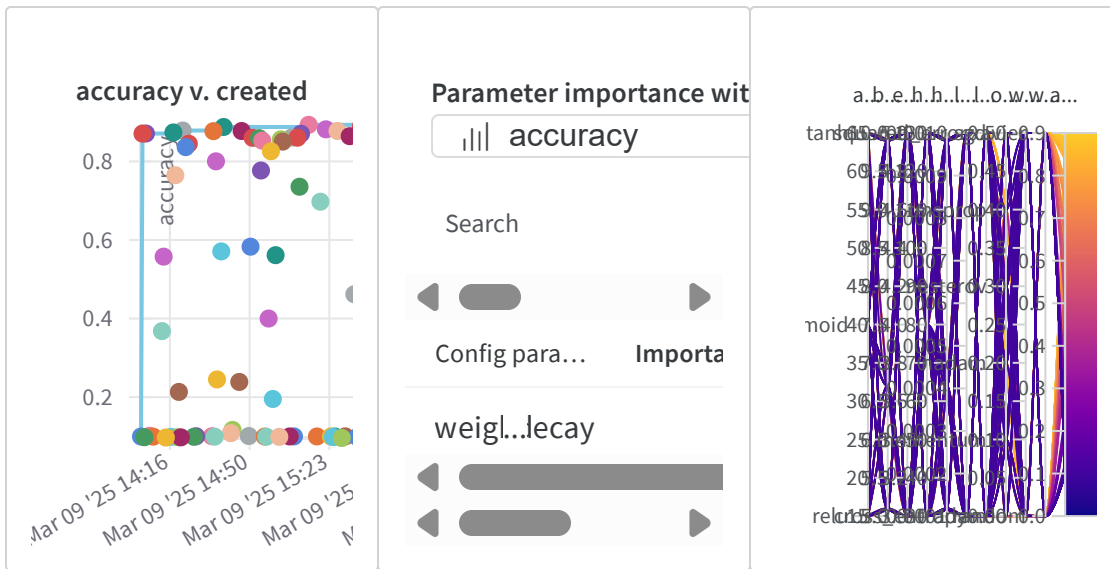
Question 8 (5 Marks)

In all the models above you would have used cross entropy loss. Now compare the cross entropy loss with the squared error loss. I would again like to see some automatically generated plots or your own plots to convince me whether one is better than the other.









Question 9 (10 Marks)

Paste a link to your github code for this assignment

Example: https://github.com/<user-id>/da6401_assignment1;

- We will check for coding style, clarity in using functions and a `README` file with clear instructions on training and evaluating the model (the 10 marks will be based on this)
- We will also run a plagiarism check to ensure that the code is not copied (0 marks in the assignment if we find that the code is plagiarized)
- We will also check if the training and test data has been split properly and randomly. You will get 0 marks on the assignment if we find any cheating (e.g., adding test data to training data) to get higher accuracy

Question 10 (10 Marks)

Based on your learnings above, give me 3 recommendations for what would work for the MNIST dataset (not Fashion-MNIST). Just to be clear, I am asking you to take your learnings based on extensive experimentation with one dataset and see if these learnings help on another dataset. If I give you a budget of running only 3 hyperparameter configurations as opposed to the large number of experiments you have run above then which 3 would you use and why. Report the accuracies that you obtain using these 3 configurations.

Code Specifications

Please ensure you add all the code used to run your experiments in the GitHub repository.

You must also provide a python script called `train.py` in the root directory of your GitHub repository that accepts the following command line arguments with the specified values -

We will check your code for implementation and ease of use. We will also verify your code works by running the following command and checking wandb logs generated -

```
python train.py --wandb_entity myname --wandb_project myprojectr
```

Arguments to be supported

Name	Default Value	Description
<code>-wp, --wandb_project</code>	myprojectname	Project name used to track experiments in Weights & Biases dashboard
<code>-we, --wandb_entity</code>	myname	Wandb Entity used to track experiments in the Weights & Biases dashboard.
<code>-d, --dataset</code>	fashion_mnist	choices: ["mnist", "fashion_mnist"]
<code>-e, --epochs</code>	1	Number of epochs to train neural network.

Name	Default Value	Description
<code>-b, --batch_size</code>	4	Batch size used to train neural network.
<code>-l, --loss</code>	cross_entropy	choices: ["mean_squared_error", "cross_entropy"]
<code>-o, --optimizer</code>	sgd	choices: ["sgd", "momentum", "nag", "rmsprop", "adam", "nadam"]
<code>-lr, --learning_rate</code>	0.1	Learning rate used to optimize model parameters
<code>-m, --momentum</code>	0.5	Momentum used by momentum and nag optimizers.
<code>-beta, --beta</code>	0.5	Beta used by rmsprop optimizer
<code>-beta1, --beta1</code>	0.5	Beta1 used by adam and nadam optimizers.
<code>-beta2, --beta2</code>	0.5	Beta2 used by adam and nadam optimizers.
<code>-eps, --epsilon</code>	0.000001	Epsilon used by optimizers.
<code>-w_d, --weight_decay</code>	.0	Weight decay used by optimizers.
<code>-w_i, --weight_init</code>	random	choices: ["random", "Xavier"]
<code>-nhl, --num_layers</code>	1	Number of hidden layers used in feedforward neural network.
<code>-sz, --hidden_size</code>	4	Number of hidden neurons in a feedforward layer.
<code>-a, --activation</code>	sigmoid	choices: ["identity", "sigmoid", "tanh", "ReLU"]

Please set the default hyperparameters to the values that give you your best validation accuracy. (Hint: Refer to the Wandb sweeps conducted.)

You may also add additional arguments with appropriate default values.

Question 10.

Configuration 1: (epochs: 10, hidden layers: 3, hidden size: 128, weight decay: 0.0005, learning rate: 1e-3, optimizer: Adam, batch size: 64, weight init: Xavier, activation: ReLU)

From the parallel coordinates plot, we can see that the highest accuracies (yellow/orange lines) tend to flow through ReLU activation, moderate-to-high batch sizes, and the Adam optimizer. The plot shows these configurations achieving better results on Fashion-MNIST. For MNIST, which contains simpler patterns than Fashion-MNIST, this configuration provides:

Sufficient model capacity: 3 hidden layers with 128 neurons each gives enough parameters to learn the digit features without excessive complexity

Efficient training: Adam optimizer with a learning rate of 1e-3 shows strong convergence paths in the visualization

Good regularization balance: The moderate weight decay (0.0005) prevents overfitting while allowing feature learning

Stable gradient flow: ReLU activation and Xavier initialization work particularly well together, as visible in the high-accuracy paths

```
wandb: View project at https://wandb.ai/da24m015-iitm/assignment_1
wandb: View run at https://wandb.ai/da24m015-iitm/assignment_1/runs/tduqp8hw
Training network with 3 hidden layers, 128 neurons, relu activation, adam optimizer...
Epoch 1/10 - Loss: 0.3037 - Acc: 0.9178 - Val Loss: 0.1427 - Val Acc: 0.9633
Epoch 2/10 - Loss: 0.1501 - Acc: 0.9633 - Val Loss: 0.1109 - Val Acc: 0.9748
Epoch 3/10 - Loss: 0.1274 - Acc: 0.9702 - Val Loss: 0.1464 - Val Acc: 0.9667
Epoch 4/10 - Loss: 0.1152 - Acc: 0.9748 - Val Loss: 0.1163 - Val Acc: 0.9753
Epoch 5/10 - Loss: 0.1072 - Acc: 0.9761 - Val Loss: 0.1171 - Val Acc: 0.9752
Epoch 6/10 - Loss: 0.1011 - Acc: 0.9788 - Val Loss: 0.1419 - Val Acc: 0.9697
Epoch 7/10 - Loss: 0.0987 - Acc: 0.9793 - Val Loss: 0.1167 - Val Acc: 0.9758
Epoch 8/10 - Loss: 0.0928 - Acc: 0.9811 - Val Loss: 0.1129 - Val Acc: 0.9760
Epoch 9/10 - Loss: 0.0910 - Acc: 0.9814 - Val Loss: 0.1731 - Val Acc: 0.9668
Epoch 10/10 - Loss: 0.0905 - Acc: 0.9820 - Val Loss: 0.1248 - Val Acc: 0.9755
Test accuracy: 97.24%
Training complete. Results logged to Weights & Biases.
```

Configuration 2: (epochs: 10, hidden layers: 4, hidden size: 64, weight decay: 0.0005, learning rate: 1e-3, optimizer: Adam, batch size: 32, weight init: Xavier, activation: ReLU)

Configuration 2 provides a different architecture balance while maintaining key successful elements:

Different capacity distribution: More layers (4) but fewer neurons per layer (64) compared to Config 1

Smaller batch size: The 32 batch size may provide more update iterations per epoch and potentially better generalization

Same strong optimizer setup: Adam at 1e-3 learning rate shows consistent performance across the visualization

Preserved regularization: Keeping the 0.0005 weight decay that appears in many high-accuracy paths

```
wandb: View project at https://wandb.ai/da24m015-iitm/assignment_1
wandb: View run at https://wandb.ai/da24m015-iitm/assignment_1/runs/bja2902j
Training network with 4 hidden layers, 64 neurons, relu activation, adam optimizer...
Epoch 1/10 - Loss: 0.3251 - Acc: 0.9081 - Val Loss: 0.1544 - Val Acc: 0.9582
Epoch 2/10 - Loss: 0.1784 - Acc: 0.9533 - Val Loss: 0.1317 - Val Acc: 0.9655
Epoch 3/10 - Loss: 0.1622 - Acc: 0.9611 - Val Loss: 0.1456 - Val Acc: 0.9677
Epoch 4/10 - Loss: 0.1577 - Acc: 0.9648 - Val Loss: 0.1120 - Val Acc: 0.9727
Epoch 5/10 - Loss: 0.1497 - Acc: 0.9669 - Val Loss: 0.1590 - Val Acc: 0.9653
Epoch 6/10 - Loss: 0.1484 - Acc: 0.9682 - Val Loss: 0.1553 - Val Acc: 0.9667
Epoch 7/10 - Loss: 0.1472 - Acc: 0.9680 - Val Loss: 0.1632 - Val Acc: 0.9672
Epoch 8/10 - Loss: 0.1448 - Acc: 0.9697 - Val Loss: 0.1403 - Val Acc: 0.9667
Epoch 9/10 - Loss: 0.1426 - Acc: 0.9712 - Val Loss: 0.2111 - Val Acc: 0.9627
Epoch 10/10 - Loss: 0.1402 - Acc: 0.9712 - Val Loss: 0.1557 - Val Acc: 0.9677
Test accuracy: 96.59%
Training complete. Results logged to Weights & Biases.
wandb:
```


Configuration 3: (epochs: 10, hidden layers: 3, hidden size: 64, weight decay: 0, learning rate: 1e-3, optimizer: RMSprop, batch size: 32, weight init: Xavier, activation: ReLU)

This configuration explores a different approach while still leveraging insights from the visualization:

Alternative optimizer: RMSprop shows some high-performing paths in the visualization and offers a different gradient adaptive approach.

No explicit regularization: Zero weight decay, relying more on the inherent regularization from batch training.

Different batch dynamics: The 32 batch size combined with RMSprop provides different convergence properties.

```
wandb: View project at https://wandb.ai/da24m015-iitm/assignment\_1
wandb: View run at https://wandb.ai/da24m015-iitm/assignment\_1/runs/m3v2n4dv
Training network with 3 hidden layers, 64 neurons, relu activation, rmsprop optimizer...
Epoch 1/10 - Loss: 0.3232 - Acc: 0.9058 - Val Loss: 0.1426 - Val Acc: 0.9582
Epoch 2/10 - Loss: 0.1699 - Acc: 0.9532 - Val Loss: 0.1344 - Val Acc: 0.9650
Epoch 3/10 - Loss: 0.1446 - Acc: 0.9635 - Val Loss: 0.1548 - Val Acc: 0.9650
Epoch 4/10 - Loss: 0.1402 - Acc: 0.9685 - Val Loss: 0.1554 - Val Acc: 0.9680
Epoch 5/10 - Loss: 0.1321 - Acc: 0.9710 - Val Loss: 0.1272 - Val Acc: 0.9718
Epoch 6/10 - Loss: 0.1327 - Acc: 0.9722 - Val Loss: 0.1377 - Val Acc: 0.9727
Epoch 7/10 - Loss: 0.1318 - Acc: 0.9733 - Val Loss: 0.1436 - Val Acc: 0.9698
Epoch 8/10 - Loss: 0.1276 - Acc: 0.9755 - Val Loss: 0.1888 - Val Acc: 0.9690
Epoch 9/10 - Loss: 0.1271 - Acc: 0.9758 - Val Loss: 0.1496 - Val Acc: 0.9717
Epoch 10/10 - Loss: 0.1257 - Acc: 0.9763 - Val Loss: 0.2813 - Val Acc: 0.9572
Test accuracy: 96.63%
Training complete. Results logged to Weights & Biases.
wandb:
```

Note: All the above results are provided with the MNIST dataset.

▸ Self Declaration

I, Rajnish Maurya_DA24M015, swear on my honour that I have written the code and the report by myself and have not copied it from the internet or other students.

Created with ❤️ on Weights & Biases.

https://wandb.ai/da24m015-iitm/assignment_1/reports/Rajnish_Maurya-DA6401-Assignment-1--VmldzoxMTcwOTAxMw